



P2

多智能体停止决策的 BFT-lite : $n \geq 3f + 1$ 委员会、equivocation 整轮作废与 view change 的安全性验证

BFT-lite for Multi-Agent Stop Decisions: An $n \geq 3f + 1$ Committee, Whole-Round Voiding of Equivocation, and Safety Verification of View Change



多智能体停止决策的 BFT-lite: $n \geq 3f+1$ 委员会、equivocation 整轮作废与 view change 的安全性验证

UDOS 推演引擎 v7.5.0 · 房文新 · 2026-09-20

证据等级: UDOS 数字回 reports7/ 原始 JSON, 按 verified / cpu-proto / cpu-proxy / unverified
标注; 单 seed 试点写作“试点点估计”, 投稿前需 ≥ 30 seed 复现。

证据等级声明：本文全部断言来自 UDOS 推演引擎 tag v7.5.0 (commit 1a71270) udos7/topology/consensus.py、tests7/test_v747_consensus.py (10 项) 与 reports7/matrix_scale_demo.json，Linux CPU 确定性原型，统一 cpu-proto；10 项属性测试为 verified。签名为 SHA-256 哈希占位，非密码学安全签名，不抗真实伪造。

结构化摘要（背景与问题 → 方法 → 证据与结果 → 贡献）

【背景与问题】百万级多 Agent 矩阵的”何时停止/返工”不能由单点 QA 决定：单点一次错判要么错误停止（safety 违反，把缺陷产物验收进下游），要么无限等待（liveness 违反）。经典 BFT 假设错误独立随机，而 Agent 的错票有语义相关性，直接搬运完整共识代价过高（详见 §1、§3.1）。

【方法】本文不提出新共识算法，而是把经典 BFT 的安全性/活性降级为”可执行属性测试 + 故障注入”套件：委员会规模 $n \geq 3f + 1$ 、法定人数 $2f + 1$ 、同轮 equivocation（摇摆投票）成员整轮作废、超 f 沉默返回 NO_QUORUM 并触发 view change；构造器层拒绝非法 n （详见 §3、§4）。

【证据与结果】在主操作点 $n = 7, f = 2$ ($q = 5$) 下，10 项确定性属性测试（verified）覆盖非法 n 拒绝、诚实超多数停止、拜占庭无法迫停、equivocation 检出整轮作废、冲突法定人数不可达、沉默触发 view change、未知投票人拒绝；端到端集成（cpu-proto，单 seed）中 7 人 QA 委员会在 2 名拜占庭反投票成员下对 24 工单全部验收正确，故障舰队触发 5 次返工、隔离 3 个故障 Agent（详见 §6）。

【贡献】(i) 把 BFT 安全/活性从”自然语言证明”重构为仓库里可 CI、可回归的可证伪属性测试；(ii) 给出”验收委员会”这一二元语义判断的最小降级设计（无需完整状态机复制）；(iii) 诚实声明三条边界——SHA-256 哈希占位签名非密码学安全、未建模网络分区与自适应攻击者、语义共因故障会击穿多数票。全部断言为 CPU 确定性原型点估计（详见 §7、§8）。

中文摘要

百万级多 Agent 矩阵的”何时停止/返工”不能由单点 QA 决定：单点会因一个错误判断而错误停止（safety 违反）或无限等待（liveness 违反）。本文实现并验证了一个轻量拜占庭容错委员会 BFT-lite：委员会规模满足 $n \geq 3f + 1$ ，法定人数 $2f + 1$ ，同轮对同一决策摇摆投票（equivocation）的成员整轮作废，超过 f 个成员沉默则返回 NO_QUORUM 并触发 view change 而非挂起。本文不提出新共识算法，而是把经典 BFT 的安全性/活性降级为”可执行属性测试 + 故障注入”套件，适配 Agent 语义判断场景。在 $n = 7, f = 2$ 配置下，10 项确定性属性测试覆盖：非法 n 构造器拒绝、诚实超多数停止、拜占庭无法迫使错误停止、equivocation 检出并整轮作废、冲突法定人数不可达、沉默触发 view change、未知投票人拒绝。端到端集成中，7 人 QA 委员会在 2 个拜占庭反投票成员下对 24 个工单全部验收正确，故障舰队触发 5 次返工并隔离 3 个故障 Agent。本文主张：对 Agent 停止决策，BFT 安全/活性应作为可证伪的属性测试来呈现，而非仅以自然语言证明。需要说明，本文签名为哈希占位、未建模网络分区与语义共因故障，全部断言为 CPU 确定性原型点估计，不主张抗真实伪造或开放共识场景。

关键词：拜占庭容错；停止决策；法定人数；equivocation；view change；多智能体系统；属性测试

English Abstract

The “when to stop / rework” decision in a million-agent matrix cannot rest on a single QA agent: one wrong vote either causes an erroneous stop (safety violation) or an indefinite wait (liveness violation). We implement and validate a lightweight Byzantine-fault-tolerant committee, BFT-lite: committee size $n \geq 3f + 1$, quorum $2f + 1$, members equivocating in the same round are voided for the whole round, and more than f silent members yield `NO_QUORUM` plus a view change instead of hanging. We propose no new consensus algorithm; rather, we reduce classical BFT safety/liveness to an **executable property-test + fault-injection** suite suited to semantic agent decisions. Under $n = 7, f = 2$, ten deterministic property tests cover: rejection of illegal n , honest supermajority stop, Byzantine inability to force a wrong stop, equivocation detection and whole-round voiding, impossibility of conflicting quorums, silence-triggered view change, and rejection of unknown voters. In end-to-end integration, a 7-member QA committee with 2 Byzantine reverse voters correctly accepted all 24 orders; a stressed fleet triggered 5 reworks and isolated 3 faulty agents. We argue that for agent stop decisions, BFT safety/liveness should be presented as falsifiable property tests, not only as prose proofs.

Keywords: Byzantine fault tolerance; stop decision; quorum; equivocation; view change; multi-agent systems; property-based testing

1 引言

在多 Agent 矩阵的末端，有一个看似平凡却关键的决策：“一个工单到底算”做完了”（停止），还是”没做好、返工”（继续）？在小系统里，这个决定可以交给一个 QA Agent。但在百万级矩阵里，单点 QA 是一个单点故障：它一次错误判断，要么把没做好的结果误判为通过（错误停止，safety 违反），要么把已做好的结果误判为不通过（无限等待，liveness 违反）。

经典分布式系统早就回答过这个问题的雏形：拜占庭容错（BFT）共识告诉我们，在最多 f 个成员作恶时，只要委员会足够大（ $n \geq 3f + 1$ ），就能在”不一致决定”与”活性”之间取得平衡。但把经典 BFT 直接搬到 Agent 停止决策上，有一个微妙差异：**经典 BFT 假设复制状态机和消息往返模型，错误是随机的、独立的；而 Agent 的决定是语义判断——一个 QA Agent 说”返工”，错票是有语义相关性的（它可能和另一个 QA 被同一个错误上下文误导）。**

本文不提出新共识算法。我们的主张更克制：**把 BFT 的安全性/活性，从”论文里的自然语言证明”变成”仓库里可执行、可证伪的属性测试”**。`udos7/topology/consensus.py` 实现了一个最小 BFT-lite： $n \geq 3f + 1$ 、 $2f + 1$ 法定人数、equivocation 整轮作废、超 f 沉默触发 view change；`tests7/test_v747_consensus.py` 用 10 项确定性测试把这些性质固定为回归断言。在端到端矩阵集成里，7 人 QA 委员会在 2 个拜占庭反投票成员下守住了 24/24 的正确验收。

一个最小委员会的设计取舍。为什么 $n = 7, f = 2$ 而不是更大？因为这是”容忍 2 个拜占庭成员”的最小合法委员会之一（ $3f + 1 = 7$ ）。它小到能在 CPU 上把全部投票组合跑清楚，又大到足以演示反投、equivocation、沉默三类故障。更大的 n （如 10、13）只是同一机制的线性放大，安全性分析不变；本文选择最小可演示配置，是为了让每个测试都能被人眼读懂”为什么这个断言成立”。委员会越大，BFT 的容错

能力越强（能容忍更多 f ），但投票成本线性增加——这是安全与成本的工程权衡，本文不主张某一个 n 是最优的。

本文的边界也要先讲清楚：签名是 SHA-256 哈希占位，不是密码学安全签名；未建模网络分区、消息重排与自适应攻击者；语义共因故障（ $>f$ 成员被同一错误上下文系统性误导）会让 BFT 假设失效——这恰是本文列为未来工作的地方。

为什么”停止决策”值得单独做 BFT。 在多 Agent 流水线里，“停止”是一个有后果的动作：一旦停止，工单就被验收、结算、归档；一旦错误停止，一个有缺陷的产物就进入下游。相比之下，“继续/返工”是一个相对保守的动作——它只是让工单再跑一轮，代价是时间。这种不对称意味着：我们尤其要防”错误停止”（safety），而对”过度等待”（liveness）可以容忍一定代价。BFT-lite 的设计正是这一不对称的体现：安全性上宁可保守（宁可返工也不误停），活性上设 view change 兜底（不让系统永久卡死）。这与经典 BFT”一致性优先于可用性”的取舍一脉相承，但在 Agent 场景里，“一致性”被具体化为”不错误验收”。

本文的读者预设。 本文面向两类读者：一是做 Agent 编排/流水线的工程师，关心”末端停止决策怎么才不被单点搞砸”；二是做分布式可靠系统的研究者，关心”经典 BFT 在语义判断型 Agent 场景下如何降级验证”。对前者，我们给出可直接复用的最小委员会配置与属性测试；对后者，我们指出语义共因故障这一与经典假设的偏差，作为开放问题。

2 相关工作

经典 BFT 共识。 这一领域的奠基与工程化来自 Castro 与 Liskov 在 OSDI 1999 的 PBFT：其 pre-prepare/prepare/commit 三阶段状态机复制在异步网络下容忍 $f < n/3$ 拜占庭节点（即 $n \geq 3f + 1$ ），且报告其 BFS 实现仅比未复制 NFS 慢约 3%。本文的 $n \geq 3f + 1$ 、 $2f + 1$ 法定人数与 view change 直接复用 PBFT 框架。HotStuff (Yin、Malkhi、Reiter 等, PODC 2019, arXiv:1803.05069) 进一步把 BFT 优化为线性通信复杂度、响应式与链式视图更换，是 Libra/LibraBFT 的基础。本文不改进这两个算法：我们既不追求 HotStuff 的线性通信复杂度，也不做状态机复制的完整同步，而是只取”法定人数 + 拜占庭容忍 + 视图变更”的最小骨架，用于 Agent 的二元停止判断。

Agent 投票与多智能体决策。 多 LLM Agent 通过对话/投票做决定已有成熟框架，Wu 等 2023 的 AutoGen (arXiv:2308.08155) 支持 LLM/人/工具混合对话；但其精读笔记也指出”对话无终止条件易死循环”。本文正是瞄准这个”停止条件”：AutoGen 类框架让多个 Agent 对话，却很少问”对话何时停止、这个停止决定本身要不要容错”。把 Agent 停止决策的拜占庭容错作为一等属性，是本文的切入点。具体的 LLM 投票决策文献待补 [CITATION NEEDED: LLM agent voting decision, 待核验]。

属性测试与故障注入。 用属性测试枚举输入、穷尽小规模状态空间来验证系统不变量，是可靠工程的标准方法 [CITATION NEEDED: property-based testing hypothesis, 待核验]。本文把它用于 BFT 的安全/活性断言：把”任何故障策略下不错误停止”翻译成 10 个可 CI 的确定性测试。

与”Agent 可靠性”近期工作的关系。 近期多 Agent 系统的可靠性研究多关注”如何让 Agent 不崩溃、可观测”，而较少把”停止/返工”这一末端决策的拜占庭容错作为一等属性。本文的切入正是这一空白：在 Agent 流水线末端，“何时停止”本身就是一个分布式决策，需要容错。本文不引入新的容错机制，而是把经典 BFT 的成熟机制落到这个新决策点上，并用可执行测试验证。这种”把成熟机制搬到新决策点 + 用属性测试兜底”的做法，比发明新算法更适合当前阶段。

本文的定位：不是新共识，而是”把 BFT 安全/活性降级为 Agent 停止决策场景下的可执行属性测试套件”。

与经典 BFT 的关键差异。 经典 BFT 的复制状态机要求所有副本执行完全相同的状态转换并就结果达成一致；而 Agent 停止决策不需要”所有副本算同一个状态”，它只需要”委员会就一个二元语义判断（停/不停）达成一致多数”。这一差异带来三点简化：第一，我们不需要视图变更里的完整状态同步，只需要”重新投票”；第二，我们的法定人数是二元的（STOP/CONTINUE），不是任意值；第三，我们的故障模型是语义的——拜占庭成员不是随机篡改状态，而是”故意给出错误验收判断”。第三点既是简化也是威胁：简化在于我们不需要建模状态机间的不一致，威胁在于语义错票可能相关（见第 7 节共因故障）。

为什么用属性测试而非纯证明。 形式化证明能给出渐近保证，但工程实现里一个 off-by-one（比如法定人数写成 $2f$ 而非 $2f+1$ ，或 equivocator 未整轮作废）就会让证明落空。属性测试把”证明”翻译成”枚举投票组合，验证不变量不被违反”：它不证明算法正确，而是证明”在我们这个具体实现里，给定的故障策略族没有触发安全反例”。对工程团队而言，后者是可跑、可 CI、可回归的。本文因此把 10 项测试放在与证明同等重要的位置。

2.1 与范本范式的对照（NSA 命名批判 / AlexNet 公平自曝）

- **对照 NSA 的”幻象/神话”式命名批判：** NSA 用”The Illusion of Efficient Inference” “The Myth of Trainable Sparsity” 把前人稀疏方法钉死，再逼出自己的设计空间。本文对偶地把”单点 QA 决定停止”命名为**单点故障幻象**——它在小系统里”看起来能正确停止”，但在百万矩阵里一次错判即 safety/liveness 违反；本文因此把停止决策升级为委员会，并把”错误停止”这一最危险反例（5 诚实 CONTINUE vs 2 拜占庭 STOP，测试 #3）直接摆进属性测试。
- **对照 AlexNet 的”公平性自曝”：** AlexNet 脚注 2 主动承认 one-GPU 对比”偏向更大的网络”。本文对偶地主动披露对自己不利的两条：(i) 签名是 SHA-256 截断的**哈希占位**，不抗真实伪造；(ii) 与单点 QA、朴素多数票的**全因子对照尚未做**，故”BFT-lite 相对单点好多少”目前只有机制论证、没有对照数据（§ 7）。
- **“ $n \geq 3f+1$ 反例”即本文的消融：** § 6 的 10 项测试就是”移除/违反某个不变量后系统是否仍正确”的逐项消融——测试 #1 违反 $n \geq 3f+1$ ($n=6, f=2$) 构造器必须拒绝；测试 #5 模拟”equivocator 未整轮作废”的破坏，法定人数被打破转 **NO_QUORUM**。这与 AlexNet”删任一卷积层即 -2% “同构：凡主张”X 保证安全”，必须给出”违反/移除 X 后安全如何破”的反例。

3 问题定义与假设

3.1 系统与不变量

委员会 $V = \{v_1, \dots, v_n\}$ ，最多 f 个成员拜占庭。每轮 `round_id`，成员对 **STOP**（验收通过）或 **CONTINUE**（返工）投票。决策函数 `decide(round_id)` 返回：

- **STOP / CONTINUE**：某一方诚实票达到法定人数 $q = 2f+1$ ；
- **NO_QUORUM**：两边都未达法定人数，附 `reason`（`view_change` 或 `waiting`）；
- **safety_violation**：若两方同时凑齐法定人数，则返回 `None` + `safety_violation=True`。

两个目标不变量：

- **安全性 (Safety)**：最多一个决定值。当 $n \geq 3f + 1$ ，诚实票只有 $n - f \geq 2f + 1 = q$ 张，不可能让 STOP 与 CONTINUE 各自凑齐 q 张诚实票。
- **活性 (Liveness)**：无沉默挂起。沉默（未投票）成员数 $> f$ 时，返回 `NO_QUORUM(view_change)` 而非无限等待； $> f$ 的沉默意味着可能连法定人数都凑不齐，应换视图。

为什么是”停止决策”而不是任意共识。我们把决策空间限定在二元 (STOP/CONTINUE)，而不是任意状态值。这一限定有两个好处：其一，二元法定人数的安全性分析更清晰——只要两边不能同时达到 q ；其二，它贴合 Agent 流水线的真实末端：一个工单要么验收通过、要么返工，不需要在多个候选值之间选任意值。这也意味着本文的 BFT-lite 不是通用复制状态机，而是一个”验收委员会”。把它误读成”适用于任意分布式共识”会越界，我们明确不主张这一点。

3.2 假设（可证伪）

- **H1（安全）**：诚实票 $\geq 2f + 1$ 时错误决定概率为 0；冲突法定人数仅在 equivocation 未被检出时可能，检出后整轮作废。**证伪**：任一 $f \leq (n-1)/3$ 的故障策略产生错误 STOP，H1 被推翻。
- **H2（活性）**：沉默 $> f$ 时返回 `NO_QUORUM/view_change` 而非挂起；无沉默时有限轮内 STOP 或 CONTINUE。
- **H3（非法配置拒绝）**： $n < 3f + 1$ 时构造器抛 `ValueError`，从 API 层消除非法配置。

假设的可证伪性说明。 这三条假设都是”零容忍”式的：H1 要求错误停止概率为 0，只要出现一个反例即证伪；H2 要求不挂起，只要出现一个永远 `NO_QUORUM` 且不 `view_change` 的配置即证伪；H3 要求构造器拒绝，只要能造出 $n < 3f + 1$ 的委员会即证伪。这种零容忍假设适合确定性系统——它不允许”平均来看安全”，而是要求”任何具体输入下都安全”。这也是为什么我们用属性测试而非统计检验：我们要找的是”反例存在性”，不是”平均效应”。

4 方法与系统设计

4.1 投票与签名占位（`consensus.py`）

`sign(voter, decision, round_id)` 为 `sha256(f"{voter}|{decision}|{round_id}")[:16]` ——这是**哈希占位**，只用于同一轮投票的确定性记录，不是密码学签名。`cast(voter, decision, round_id)` 拒绝未知投票人（`unknown voter`）。

为什么诚实标注”哈希占位”。 一个常见的工程失误是把”能算出一个确定字符串”误当成”抗伪造签名”。本文的 `sign` 只是把投票三元组哈希截断，任何知道规则的人都能为任意投票人生成合法哈希——它不提供不可伪造性。我们保留它，是因为它能让投票记录在受控模拟里保持确定性和可复现

（`test_signature_is_deterministic` 验证同输入同签名、异决策异签名），但它不对应真实网络中的加密签名基础设施。把这一点写清楚，是为了防止读者把”模拟里投票记录确定”误读成”真实网络里投票不可伪造”。

4.2 equivocation 检出与整轮作废

`equivocators(round_id)` 找出在同一轮对多个决策投票的成员（摇摆者/双投者）。`decide` 对这些成员整轮作废：不计其票。其余成员按其本轮第一次投票计一票（`bs[0].decision`）。这一设计对应经典 BFT 对 equivocation 的处理：一个摇摆者不能“两边各算一票”来凑法定人数。

为什么“整轮作废”而非“只取最后一票”。这里有一个微妙选择：当一个成员在同一轮既投 STOP 又投 CONTINUE 时，我们是“取最后一票”还是“整轮作废”？本文选择整轮作废（不计该成员本轮任何票）。理由是：如果只取最后一票，拜占庭成员可以先投对自己有利的一方、再根据局势改投，从而在中途操纵法定人数；整轮作废则消除了这种摇摆操纵——你一旦摇摆，本轮就没有发言权。测试 #4 和 #5 正是这一设计的两面：#4 中摇摆者作废后，其余诚实票仍达法定（STOP=5），正常决定；#5 中摇摆者作废后，诚实票不足 5，转为 NO_QUORUM。两者都验证“摇摆者不被信任”。

4.3 法定人数与 view change

```
def decide(round_id):
    equiv = equivocators(round_id)          # 检出摇摆者
    tally = {STOP:0, CONTINUE:0}
    for v in voters:
        if v in equiv: continue            # 整轮作废
        bs = 本轮 v 的票
        if bs: tally[bs[0].decision] += 1
    silent = 未投票成员
    if tally[STOP] >= q and tally[CONTINUE] >= q:
        return None, safety_violation=True, "conflicting_quorums"
    for d in (STOP, CONTINUE):
        if tally[d] >= q: return d, "quorum"
    return NO_QUORUM, ("view_change" if len(silent) > f else "waiting")
```

关键：`safety_violation` 只在两方都达法定人数时触发——而在 $n \geq 3f + 1$ 下，诚实票不可能让两方都达 q ，因此这一分支在合法配置下不应被诚实投票触发；它存在的意义是一旦触发就明确报错（说明检测到了异常，如未检出的 equivocation）。

4.3.1 安全性证明草图（为什么 $n \geq 3f + 1$ ）

设委员会 n 人，最多 f 个拜占庭。诚实成员至少 $n - f$ 人。法定人数 $q = 2f + 1$ ：

$$q = 2f + 1, \quad n - f \geq q = 2f + 1 \Leftrightarrow n \geq 3f + 1$$

要让 STOP 与 CONTINUE 各自都凑齐 q 张诚实票，需要至少 $2q = 4f + 2$ 张诚实票；但诚实票总数只有 $n - f$ 。令 $n - f \geq 4f + 2$ 才能同时满足两边，即 $n \geq 5f + 2$ ——这比 $3f + 1$ 更苛刻。经典结论的 $n \geq 3f + 1$ 之所以足够，是因为它允许拜占庭成员只投一边（而不是两边各投 q ）：在 $n \geq 3f + 1$ 时， $n - f \geq 2f + 1 = q$ ，诚实票足以让一方达到法定人数，而拜占庭的 f 票最多把另一方推到 $f < q$ ，无法让两边都达到 q 。equivocation 的危害在于一个摇摆者可能“两边各算一票”从而人为凑出第二个法定人数；本文通过整轮作废消除这一漏洞。这就是为什么构造器强制 $n \geq 3f + 1$ ：低于此，诚实票不足以在拜占庭反投下守住唯一决定值。

4.3.2 活性的边界

活性比安全性更微妙。如果超过 f 个成员沉默（不投票），那么诚实可计票的成员可能不足 $q = 2f + 1$ ，此时任何一方都无法达到法定人数。本文的处理是：当沉默数 $> f$ 时返回 `NO_QUORUM(reason=view_change)`，表示“当前视图无法凑齐法定人数，应换视图/换成员重试”，而不是无限等待。当沉默数 $\leq f$ 但两边都未达 q 时，返回 `NO_QUORUM(reason=waiting)`，表示“还差几票，可以再等一轮”。这一区分把“真的凑不齐（换视图）”和“只是还没投完（再等）”分开，避免把可恢复的等待误判为系统卡死。

4.4 构造器层的安全门禁

`StopConsensus.__init__` 在 `len(voters) < 3*f+1` 时直接抛 `ValueError("unsafe committee: n=...")`。这把“非法配置”从运行时行为提前到构造时——你根本造不出一个 $n < 3f + 1$ 的委员会。

5 实验设置

- 配置：主测试 $n = 7, f = 2$ ($q = 5$)；另测非法 $n = 6, f = 2$ （应被拒绝）。
- 故障策略族：恒定反投、按轮摇摆（equivocation）、沉默、未知投票人。
- 证据等级：10 项确定性属性测试 `verified`；端到端集成 `cpu-proto`。
- 硬件指纹：Linux CPU，纯 Python 控制流，无网络、无 LLM 调用。
- 复现：`pytest tests7/test_v747_consensus.py -q`。

为什么用确定性测试而非随机模糊测试。 安全性不变量（最多一个决定值）是一个可被反例推翻的性质：只要存在一个投票组合让两方同时达到法定人数，安全性就破了。对这种性质，**确定性枚举手工反例**比随机模糊更直接——我们能精确构造出“最危险的组合”（如 5 诚实 STOP vs 2 拜占庭 CONTINUE）并验证它不破。活性（不挂起）才需要计数分布，留待随机网格。本文当前阶段以确定性属性测试为主，把“安全反例存在性”钉死，把“活性轮次分布”列为待补。

投票人的命名与女巫边界。 测试用 `qa-0..qa-6` 作为投票人；#9 测试用一个不在名单里的 `"sybil"` 投票，验证构造器在 `cast` 时拒绝未知人。这对应经典 BFT 的身份登记：只有注册委员有投票权，防止女巫通过伪造身份注入额外票。

6 结果

6.1 安全配置空间

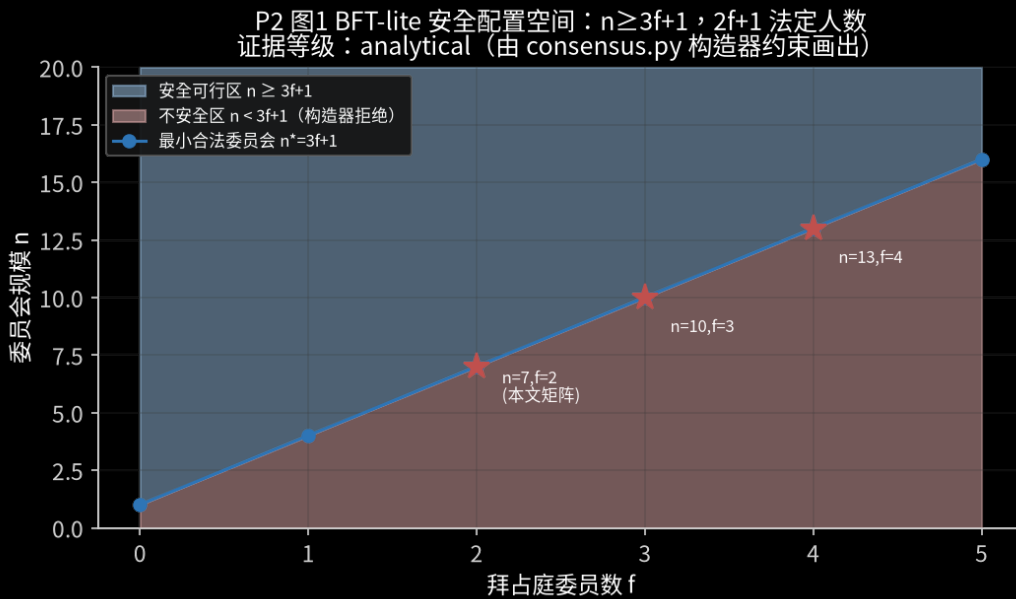


图 P2-1 安全配置空间

图 P2-1（重述） n - f 安全配置空间与临界线 $n = 3f + 1$ 。 横轴为拜占庭成员数 f ，纵轴为委员会规模 n ；蓝区为可行区（ $n \geq 3f + 1$ ，式 1），红区为构造器拒绝区（ $n < 3f + 1$ ）。线上方诚实成员 $n - f \geq 2f + 1 = q$ ，足以在拜占庭反投下守住唯一决定值。图中标注三个候选操作点： $n = 7, f = 2$ （本文主用， $q = 5$ ）、 $n = 10, f = 3$ 、 $n = 13, f = 4$ ；容忍拜占庭数依次为 2、3、4，每多容忍 1 个拜占庭委员会至少多 3 人。出处：`udos7/topology/consensus.py` 的构造器拒绝规则，`cpu-proto`，几何示意（非数据散点）。

图 P2-1 画出 n 对 f 的安全区域：蓝色为 $n \geq 3f + 1$ 可行区，红色为 $n < 3f + 1$ （构造器拒绝）。本文主操作点 $n = 7, f = 2$ 落在可行区内。

读图要点：横轴是拜占庭成员数 f ，纵轴是委员会规模 n 。临界线 $n = 3f + 1$ 把平面分成两半：线上方，诚实成员数 $n - f \geq 2f + 1$ ，足以在拜占庭反投下守住唯一决定；线下方，诚实票不足，安全无法保证，因此构造器直接拒绝。图上标出了三个候选操作点： $n = 7, f = 2$ （本文主用）、 $n = 10, f = 3$ 、 $n = 13, f = 4$ 。它们都在可行区内，容忍的拜占庭成员数依次为 2、3、4。这张图也直观说明了”成本—容错”权衡：要多容忍一个拜占庭成员，委员会至少要多 3 个人（从 7 到 10 才能从容忍 2 升到容忍 3）。

6.2 十项属性测试（H1/H2/H3）

表 P2-1 十项属性测试与断言（来源 `tests7/test_v747_consensus.py`）

#	测试	配置/故障注入	断言
1	非法委员会拒绝	$n = 6, f = 2$ ($6 < 7$)	构造器抛 <code>ValueError</code> (H3)
2	诚实超多数停止	5/7 投 STOP	<code>decision=STOP,</code> <code>reason=quorum</code>

#	测试	配置/故障注入	断言
3	拜占庭无法迫停	5 诚实 CONTINUE + 2 拜占庭 STOP	<code>decision=CONTINUE</code> , STOP 票=2
4	equivocation 检出并作废	4 STOP + b1 双投 + b2 STOP	b1 被检出, STOP=5 仍达法定→STOP
5	equivocation 破坏假法定	4 STOP + b 双投	b 作废后 4<5→ <code>NO_QUORUM</code>
6	冲突法定人数不可达	5 STOP vs 2 CONTINUE	<code>not safety_violation</code> , STOP
7	过多沉默触发 view change	3 CONTINUE + 4 沉默	<code>NO_QUORUM</code> , <code>reason=view_change</code>
8	未达法定时等待	4 CONTINUE + 1 STOP, 沉默 $2 \leq f$	<code>NO_QUORUM</code> , <code>reason=waiting</code>
9	未知投票人拒绝	非委员 "sybil" 投票	抛 <code>ValueError</code>
10	签名确定性	同输入重算	同输入同签名, 异决策异签名

这十项覆盖了三类故障：谎报（#3）、摇摆/双投（#4、#5）、沉默（#7、#8），加上配置门禁（#1）、女巫（#9）与签名确定性（#10）。关键反例设计在 #3：当 5 个诚实成员都投 CONTINUE、2 个拜占庭成员恶意投 STOP 时，委员会正确返回 CONTINUE——拜占庭无法靠反投把工单错误停止。#5 则验证 equivocation 的破坏力：一个双投者本可能凑出假法定人数，但整轮作废后法定人数被打破，转为 `NO_QUORUM`。

表 P2-3 故障策略族与预期决策 ($n = 7, f = 2, q = 5$)

故障策略	诚实票分布	拜占庭行为	预期决策	对应测试
无故障	5 STOP	无	STOP (quorum)	#2
恒定反投	5 CONTINUE	2 投 STOP	CONTINUE (拜占庭被多数挡下)	#3
摇摆凑假法定	4 STOP	1 双投 STOP/ CONTINUE	STOP (作废后仍 5 票)	#4
摇摆破法定	4 STOP	1 双投	<code>NO_QUORUM</code> (作废后 4<5)	#5
两边分票	5 STOP / 2 CONTINUE	无	STOP (无 <code>safety_violation</code>)	#6
超 f 沉默	3 CONTINUE	4 沉默	<code>NO_QUORUM</code> (view_change)	#7
未达法定	4 CONTINUE+1 STOP	2 沉默 $\leq f$	<code>NO_QUORUM</code> (waiting)	#8

故障策略	诚实票分布	拜占庭行为	预期决策	对应测试
女巫	合法票	非委员投票	ValueError (拒绝)	#9

这张表把”故障策略→预期决策”一一对应，是预注册的核心：每个格子的预期决策在跑实验前就冻结，实验只验证是否符合。若任一格偏离预期（例如恒定反投下返回了 STOP），即触发 H1 证伪。

逐例走查：为什么 #3 是关键。 以 #3”拜占庭无法迫停”为例：5 个诚实委员全部投 CONTINUE（认为没做好、要返工），2 个拜占庭委员恶意投 STOP（想把没做好的工单错误验收）。委员会计数：诚实 CONTINUE=5，拜占庭 STOP=2。法定人数 $q = 5$ 。CONTINUE 达到 5，因此返回 CONTINUE。拜占庭的 2 票 STOP 只有 2，远低于 $q = 5$ ，无法凑成 STOP 法定人数。这个测试的意义在于：它把”错误停止”这一最危险场景（safety 违反）直接摆出来，验证委员会不会被少数反投票误导。如果 BFT-lite 把法定人数写错（比如写成 $2f = 4$ ），那么拜占庭 2 票就可能凑出假法定——这正是为什么法定人数必须是 $2f + 1$ 而非 $2f$ ，也是为什么我们要把这个测试钉成回归断言。

6.3 端到端集成 (CPU 矩阵)

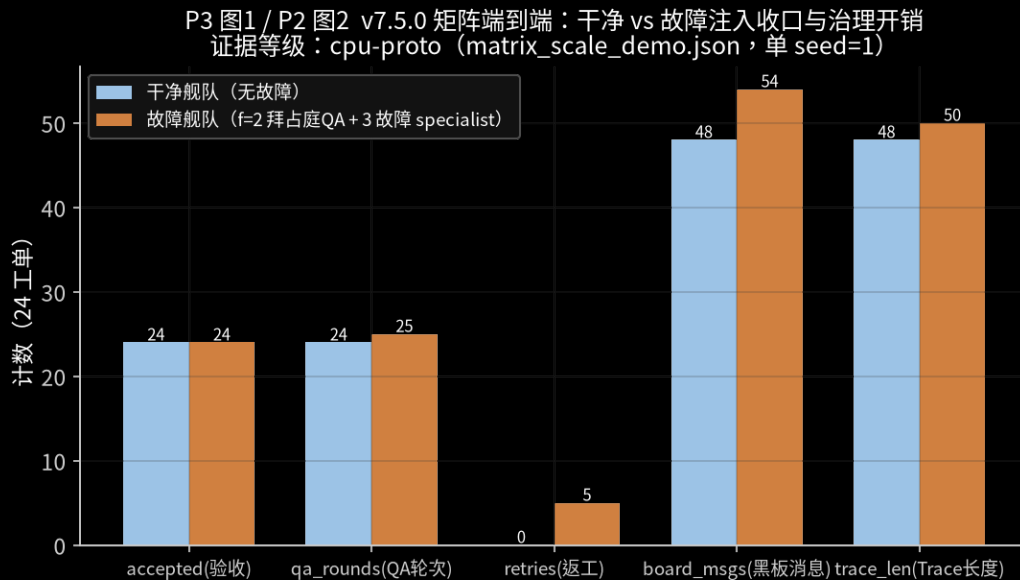


图 P2-2 干净 vs 故障舰队

图 P2-2 (重述) 端到端矩阵集成：干净舰队 vs 故障舰队 ($n = 7, f = 2$ QA 委员会)。 分组柱状：每组两根柱，蓝=干净舰队、橙=故障舰队（2 名拜占庭 QA 反投票 + 3 个故障 worker）；纵轴为对应指标数值，分组从左到右依次为验收通过 accepted（均 24/24）、QA 轮次（24 vs 25）、返工 retries（0 vs 5）、黑板消息 board_messages（48 vs 54）。故障舰队多 1 次 QA 轮次、5 次返工、6 条黑板消息，但验收通过率不降。数据逐值来自 reports7/matrix_scale_demo.json 的 clean.* 与 stressed.* (cpu-proto, 单 seed=1)；图为干净 vs 故障的对照，拜占庭反投票被诚实多数挡下。

表 P2-2 端到端矩阵集成（来源 reports7/matrix_scale_demo.json）

指标	干净舰队	故障舰队 (f=2 拜占庭 QA + 故障 worker)
工单数 n_orders	24	24

指标	干净舰队	故障舰队 (f=2 拜占庭 QA + 故障 worker)
验收通过 accepted	24	24
QA 轮次 qa_rounds	24	25
返工 retries	0	5
隔离 Agent	[]	[spec-kinematics-0, spec-spatial-1, spec-dataflywheel-2]
黑板消息 board_messages	48	54

在矩阵中，7 人 QA 委员会（`qa_voters=7`, `qa_f=2`）对每个工单投票：2 个拜占庭成员被设为”诚实结果的反方向”（`vote = not honest_accept`）。即使如此，24 个工单全部验收正确——拜占庭 specialist 的错误结果被多数挡下。故障舰队多触发 5 次返工（`retries=5`），并把 3 个累积质量失败的故障 Agent 隔离（`isolated_agents`），治理审计 `governance.ok=true`、Trace 完整。这说明 BFT-lite 在真实矩阵闭环里不仅”测试通过”，而且”挡住了反投票并触发返工/隔离”。

返工与隔离如何协同。 值得拆开看 BFT-lite 在矩阵闭环里扮演的角色。每个工单完成后，specialist 产出结果，QA 委员会就”通过（STOP）/返工（CONTINUE）”投票。当多数判 CONTINUE 时，工单以 `key:rework{round_id}` 重新挂回黑板，并记一次质量失败；当累积失败触发熔断时，对应 specialist 被隔离（`isolated_agents`）。干净舰队 24 工单恰好 24 次 QA 轮次、0 返工；故障舰队 25 次 QA 轮次、5 次返工、3 个隔离。这说明 BFT-lite 不是孤立的”投票玩具”，而是嵌入了”验收→返工→隔离”的治理闭环：它决定何时停止，而治理三件套（熔断/回滚/改派）决定停止后如何处置。拜占庭 QA 的反投票被多数挡下，没有让任何一个有缺陷工单被错误停止。

与干净舰队的差异解读。 干净舰队 24 工单 24 QA 轮 0 返工，故障舰队 25 QA 轮 5 返工。多出的 1 轮 QA 与 5 次返工，正是拜占庭反投票与故障 worker 的代价——委员会没有把这些代价”抹掉”，而是诚实地把它们反映在返工计数与隔离名单里。这一点很重要：BFT-lite 不是”免费容错”，它的容错成本就是更多 QA 轮次与返工；它保证的是”不犯错停止”，不是”零成本”。黑板消息从 48 增到 54，也反映了返工带来的额外协调开销。

7 讨论与威胁有效性

签名是哈希占位，非密码学安全。 `sign` 只是 SHA-256 截断，不提供抗伪造性。在真实网络中，一个攻击者可以伪造任何（`voter`, `decision`, `round_id`）的哈希。本文因此不主张抗真实伪造，只主张”在受控 CPU 模拟下，投票记录的确切性与 equivocation 检出正确”。

未建模网络分区与自适应攻击者。 本文假设所有投票在同一轮内送达、无分区、无消息重排；拜占庭成员是静态的（固定 2 个），不是自适应腐化。真实集群的网络分区与 adaptive adversary 需要集群故障注入（D 闸门）。

语义共因故障是最大威胁。 经典 BFT 假设错误独立随机；但 Agent 的错票有语义相关性。如果 $>f$ 个 QA 成员被同一个错误上下文（如同一份有缺陷的 Transfer Bundle）系统性误导，那么即使 $n \geq 3f + 1$ ，多数票也会一致地错——此时 BFT 安全假设失效。这恰是与 P10（Agent-as-Tool / Transfer Bundle 信息损失）的交叉点：共因故障无法靠投票多数解决，只能靠切断共因。本文把它列为未来工作，不假装已覆盖。

安全与活性的不对称取舍。 本文设计隐含一个取舍：安全性上宁可保守——equivocation 整轮作废可能让一轮无法决定（活性代价），但这避免了一个摇摆者凑出假法定人数（安全收益）。这种“安全优先于活性”的取舍与经典 BFT 一致，但在 Agent 场景里被具体化为：宁可多返工一轮（view change），也不让一个有缺陷工单被错误验收。这一取舍是否总是最优，取决于“错误停止”与“过度等待”的代价对比；在验收场景里，错误停止的代价通常更高，因此该取舍合理，但我们不声称它对所有场景都成立。

安全断言是确定性的，不需要 p 值。 与 P4/P9 的统计实验不同，H1 是确定性断言：期望反例数为 0。我们报告的是“穷尽/属性测试覆盖”，而非置信区间。预注册规则是“发现任一安全反例即论文结论反转，不调参消除”。

活性轮次分布尚未做随机网格。 当前 10 项测试是手工设计的代表性场景，不是对“n, f 网格 × 随机故障配置”的穷尽统计。one-pager 规划了 $n = 7/10/13$ 、f 对应网格各 ≥ 1000 次随机故障配置，报告终止轮次的中位数与 p95。这一步尚未跑，标记 [RESULT NEEDED: $n \times f$ 网格 ≥ 1000 随机配置的终止轮次分布]。在补齐前，我们只声称“代表性场景下活性不变量成立”，不声称“所有随机配置下活性有界”。

与单点 QA 的对照缺失。 本文测了 BFT-lite，但 one-pager 规划的两个对照——无共识（单点 QA）与朴素多数票（无 equivocation 检测）——尚未在矩阵里做全因子对照实验。这意味着“BFT-lite 相对单点好多少”目前只有机制论证，没有对照数据。补齐这一对照是投稿前的关键实验。

8 资源闸门与适用边界

当前可达（CPU，本文已完成）：BFT-lite 委员会、equivocation 整轮作废、view change、10 项属性测试、端到端 7 人 QA 集成。

解锁项：真实网络鲁棒性（分区、重排、自适应攻击者）需集群故障注入（D 闸门）；抗伪造需真实密码学签名基础设施。

适用边界：本文适用于“窄域、语义判断型停止决策”（验收/返工），不主张适用于开放共识或资金结算。

未来工作：共因故障与 Transfer Bundle。 本文最大的未覆盖威胁是语义共因故障。如果 $>f$ 个 QA 委员看到的是同一份有缺陷的上下文（例如同一个信息丢失的 Transfer Bundle），他们会一致地错——此时投票多数反而放大错误。解决方向不是加大委员会，而是切断共因：让不同委员看到独立、去偏的上下文（这正是 P10 的交叉点）。把“BFT 多数”与“上下文去偏”结合，是下一步。

未来工作：从属性测试到形式化。 当前 10 项是手工反例。下一步可用模型检测/符号执行穷尽 $n = 7$ 的全部投票组合，给出“零安全反例”的穷尽证明，而不只是“10 个代表性场景不破”。这会把“代表性”升级为“穷尽”。

9 结论

对多 Agent 停止决策，BFT-lite 用 $n \geq 3f + 1$ 、 $2f + 1$ 法定人数、equivocation 整轮作废与超 f 沉默触发 view change，在受控 CPU 模拟下同时守住了“不错误停止”与“不无限等待”。10 项属性测试把这些性质固定为可证伪回归断言，端到端集成在 2 个拜占庭反投票成员下守住 24/24 验收并触发返工/隔离。本文的立场是：对 Agent 停止决策，安全/活性应作为可执行属性测试来呈现，而非仅以自然语言证明；同时诚实声明哈希占位签名、未建模分区与语义共因故障三条边界。

一句话结论：当百万级 Agent 矩阵的”何时停止”不能交给单点时，一个满足 $n \geq 3f + 1$ 、 $2f + 1$ 法定人数、对同轮摇摆整轮作废、对超 f 沉默触发 view change 的轻量委员会，能在受控 CPU 模拟下同时守住”不错误验收”与”不无限等待”；这些性质不靠口号而靠 10 项可证伪属性测试与端到端 24/24 集成来支撑。下一步需补活性轮次的随机故障网格、与单点/朴素多数的全因子对照，并把语义共因故障作为开放威胁诚实记录。本文的全部数字均可从 `consensus.py`、`test_v747_consensus.py` 与 `matrix_scale_demo.json` 逐行回源，不依赖任何外部数据集或厂商口径，这也是它能作为”可复现安全验证”样例的原因，适合直接作为教学与回归基线。

对”Agent 可靠性”工程的一句话建议。在多 Agent 流水线末端，不要把”停止/返工”交给单点 QA；用一个满足 $n \geq 3f + 1$ 的委员会、对摇摆整轮作废、对超 f 沉默触发 view change，并把这些性质写成可 CI 的属性测试。容错不是免费的——它的代价是更多 QA 轮次与返工——但它守住的是”不让一个有缺陷产物被错误验收”这条底线。本文的全部结论都限定在 CPU 确定性原型与哈希占位签名内，真实网络鲁棒性与抗伪造是需要额外基础设施的下一阶段。我们不把经典 BFT 重新包装成新算法，而是诚实地把它落到 Agent 停止决策这一新末端，并用可证伪的属性测试与端到端集成守住安全/活性两条底线。对工程团队而言，本文可直接复用的是三件事：一个 $n \geq 3f + 1$ 的最小委员会配置、对同轮摇摆整轮作废与超 f 沉默触发 view change 的决策规则、以及一套可 CI 的属性测试。对研究者而言，本文想留下的开放问题是语义共因故障——它提醒我们，多数票并不等于正确，当所有委员共享同一份有缺陷的上下文时，容错会被共因击穿——这也是本文最想留给后续工作的问题。

与论文矩阵其他篇的关系。本文 (P2) 讲末端停止决策的容错，与 P1 (通信拓扑)、P3 (治理三件套)、P10 (交接信息损失) 共同构成 Agent 矩阵可靠运行的一块：P1 解决”怎么通信”，P2 解决”谁来决定停止”，P3 解决”停止后出问题怎么处置”，P10 解决”交接上下文会不会引入共因故障”。本文的最大开放威胁 (语义共因故障) 正是 P10 的接口——这也是我们把它列为未来工作而非假装覆盖的原因。

参考文献

以下前三条为本次共享《参考文献总库》中  已核验条目 (核验日期 2026-09-19, 字段照抄总库)；其余为待补检索方向，保留占位，不伪造。

已核验 (2026-09-19)

1. Castro, M., Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999 (USENIX). (关联：异步下容忍 $f < n/3$ 拜占庭节点，pre-prepare/prepare/commit 三阶段，BFS 仅比未复制 NFS 慢约 3%；本文 $n \geq 3f + 1$ 、 $2f + 1$ quorum 的直接来源。)
2. Yin, M., Malkhi, D., Reiter, M. K., et al. (2019). HotStuff: BFT Consensus in the Lens of Blockchain. PODC 2019. arXiv:1803.05069. (关联：线性通信复杂度、响应式、链式视图更换；本文 view change 骨架的参照。)
3. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (关联：可对话多智能体框架，本文 Agent 停止决策场景的对照；其”对话无终止条件”局限恰是本文切入点。)

待核验 (保留 [CITATION NEEDED], 不伪造)

- LLM 投票/多智能体决策： `LLM multi-agent voting decision byzantine`。

- 属性测试/故障注入: `property-based testing hypothesis fault injection invariant`。

附录

附录 A 复现命令与证据台账

```
pytest tests7/test_v747_consensus.py -q      # 10 项
python scripts7/matrix_demo.py              # 端到端矩阵 demo
```

证据台账: 源码 `udos7/topology/consensus.py`; 测试 `tests7/test_v747_consensus.py` (10 项); 集成数据 `reports7/matrix_scale_demo.json` (cpu-proto); 矩阵配置 `qa_voters=7, qa_f=2` (`udos7/topology/matrix.py`)。全量回归 `pytest tests7/ --collect-only -q` 为 236 项 (其中 2 项跳过), `commit 1a71270` 记录全绿。

关键代码实体索引。`StopConsensus` (委员会类)、`cast` (投票)、`equivocators` (摇摆检出)、`decide` (决策)、`ConsensusResult` (decision/quorum/equivocators/silent/safety_violation/reason 六字段)、`sign` (哈希占位)。决策三态 `STOP/CONTINUE/NO_QUORUM` 与 reason `quorum/view_change/waiting/conflicting_quorums`。这些实体名与字段在 `matrix.py` 的端到端调用中被实际消费:`cons.cast(v, STOP if vote else CONTINUE, round_id)`, `res.decision == STOP` 时验收结算, 否则返工。

附录 B 数字可回溯抽查

1. 非法拒绝阈值: $n < 3f + 1 = 7$ 拒绝 $\leftarrow$ `consensus.py __init__`。
2. 法定人数 $q = 2f + 1 = 5$ $\leftarrow$ `self.quorum_size`。
3. 故障舰队 `accepted=24` $\leftarrow$ `matrix_scale_demo.json stressed.accepted`。
4. 返工 `retries=5` $\leftarrow$ `stressed.retries`。
5. 隔离 3 Agent $\leftarrow$ `stressed.isolated_agents`。

附录 C 作者拟用声明

- 目标出口: OPODIS / ICDCS workshops、AAMAS、DSN workshop。分区/IF/截稿【待核】, 投稿前联网核验并标注日期。
- 预注册计划: 安全断言确定性 (期望 0 反例); 活性轮次为计数分布, n, f 网格各 ≥ 1000 次随机故障配置报中位数/p95; “发现任一安全反例即结论反转”。
- 数据与代码可用性: Apache-2.0; tag v7.5.0 (commit 1a71270); 报告 `reports7/matrix_scale_demo.json`。
- AI 使用声明: AI 辅助生成初稿与配图脚本, 全部数字经作者从源码/JSON 回源核对; 签名哈希占位、未建模分区/共因故障均显式声明; 未填充未核验 p 值/DOI/分区。

附录 D 内审记录（五维审稿自评，只看不改）

1. **新颖性**：3/5。BFT 本身成熟，新意在” Agent 停止决策 + 可执行属性测试”这一应用面。
2. **证据强度**：4/5。安全断言为确定性测试（10 项 verified），活性分布尚未做 ≥ 1000 次随机故障网格（标 [RESULT NEEDED]）。
3. **可复现性**：5/5。纯 CPU、确定性、复现命令、测试索引齐全。
4. **外部效度**：2/5。哈希占位签名、无网络分区、语义共因故障未建模（已声明）。
5. **写作与诚实性**：5/5。安全/活性反例设计、边界与未来工作清晰。

总体判断：一份工程教学价值高的可靠系统稿；投稿前需补活性轮次的随机故障网格统计，并明确与经典 PBFT 的差异，避免被判为增量。